Further simplification of DiophantineConsistency Lean proofs

Source review and new proposals after the recent commits

26 October 2026 Second review

Review pin `b310142330df89e48413c8fce62e7de223e1c7d5`

Main conclusion. The recent commits add a new layer of exact word, cell-cycle and Wang-boundary proofs; they do not simplify the older proof bodies. The best next edits are local: reuse an existing raw-table decoder, fuse two tree-decoder invariants, and specialize APIs that the new modules already provide. These changes are independent of the previously submitted large circuit, bound, carry and representation refactors.

ID	Proposed change	First acceptance test
A1	Replace the parent-code reconstruction induction with total decoding and `check_lists`.	Same raw-input theorem
A2	Prove tree quotation and depth together; retain both old theorem names as projections.	One decoder traversal
C1	Use `Finset.sum_le_one_iff` for the all-zero or selected-index dichotomy.	Empty type still works
C2	Specialize `binaryWord_prefix` directly for offsets 19 and 20.	Exact low-bit formulas
C3	Move the existing binary-word congruence lemma to the definition layer.	No import cycle
C4	Derive zero-base propagation from `propagate_from`.	Reorder before reuse
W1	Expose an indexed north-boundary view for five classification proofs.	Same literal tile cases
S1–S2	Make one CRT congruence explicit; share binary composition depth bookkeeping.	Same certificate and bounds

What is new. All proposals were compared with the five ZIP packets at `docs/incoming` and the sixth packet routed to the research archive. Section 2 identifies the nearest excluded ideas and the specific obligation each new proposal removes. The packet contains no new recommendation to repeat the old circuit certificates, general scalar bounds, sum-of-squares infrastructure, tuple packaging or scanner frameworks.

Measured text only. A1 and A2 replace 103 nonblank declaration lines with 49, counting the shared theorem and wrappers. The six targeted current-module proof bodies in C1–C4 total 78 versus 31 nonblank lines before compatibility alias overhead. These are static draft counts, not compiled savings or performance measurements.

Validation boundary. All new Lean code is **uncompiled**. The environment has no Lean/Lake toolchain; no installation was attempted. Source bytes and source anchors were verified, and the report was built and visually reviewed. No Lean elaboration, kernel replay, fresh axiom audit or benchmark is claimed. The source snapshot is unchanged and no upstream change was published.

Reading order. Commit context and novelty come first; each proposal then gives the source, mechanism, candidate and adoption gate. Full Lean files, patches, measurements and provenance accompany this TeX/PDF report.

1 What the recent commits change

The comparison is from the previous review pin `60e9c5d620c6c387d932876cfa2d9376f64ea56c` to the present immutable pin. The previous pin is the merge base: the repository is ahead by 21 commits and behind by zero. The head commit is timestamped 26 October 2026 at 16:18:46 UTC. The complete compare and per-commit metadata are included in `provenance/`.

Net change	Consequence for this review
28 new active Lean modules	14 proof/definition modules and 14 matching Checks modules. These are new review surfaces, not replacements of old proofs.
14 new root imports	The sole changed pre-existing active Lean file is the root facade; no existing active proof body changes in this interval.
3,307 added active Lean lines	3,293 are in new modules, 14 are imports; zero deletions.
60 research-archive files	Prior proposals were placed, not adopted as active proof simplifications.
29 review documents	New implementation reviews supply context, but their build receipts are upstream claims, not reproduced validation here.

1.1 The new proof layer in four stages

Actual arithmetic words. Commits `3951d56119`, `876f6b409b` and `7e9e9235e5` connect copy parity, whole-cell alignment and literal neighboring bits. C2 and C3 simplify the proof plumbing around these exact formulas; neither changes the formulas nor substitutes residue-only geometry.

Selection and spatial propagation. The classification, cell-cycle, true-color and occupation commits `df7f0c9c9b`, `a54685015f`, `2207b481bc` and `db86282477` connect local tile possibilities with exact occupation. C1 removes a manual finite-sum argument; C4 removes a special-case induction already subsumed by the general propagation theorem.

Cyclic word semantics. Commits `98bfa794dd`, `cf0b056cc8` and `bfdd368be9` define the marked word, transport physical shifts through its canonical coordinate, and map actual positive polynomial zeros to it. None of the proposals alters that map, the ambient modulus, the chosen coordinate or the word's temporal orientation.

Boundary classification. Commits `9cef0257dd` and `73d3d8e9c0` classify exact boundary colors and seam behavior. W1 factors repeated north-color elimination without changing the literal initializer or adding any strip, run, halting or tiling premise.

1.2 Evidence and limits

The snapshot contains 826 selected files, including all 574 active `.lean` files in the target Lean subtree. Each file's bytes were checked against its pinned Git blob identity and recorded with SHA-256: all 826 checks pass. Of these, 766 unchanged files were reused only after a fresh object-identity check; 60 were fetched by immutable blob URL. Incoming ZIPs have a separate six-packet byte verification. The audit is a source-level result, not proof of elaboration or mathematical correctness.

2 Novelty against all submitted packets

The exclusion corpus comprises all five ZIPs present in `docs/incoming` at the review pin and the sixth retired packet explicitly linked by that directory's README. The six manuscripts and their technical notes were inspected, not just their filenames. Their 75 proposal families are recorded in `incoming_audit/EXCLUSION_CATALOG.md`; the source key A–F and archive identities appear at the end of this report.

Decision rule. Renaming a helper or changing tactics does not make an old proposal new. The same mechanism at the same target or announced consumer family is excluded. Generic similarity in using induction, a named library API or finite classification does not exclude a different theorem that removes a different obligation. The following matrix records that boundary explicitly.

New	Closest prior scope	Distinct obligation removed
A1	EX001 list-lookup inductions; EX002 Option-map injectivity; EX007–009 scanner iteration/history folds	Exact re-encoding of an arbitrary raw parent code is supplied by existing total decoding. No lookup lemma or scanner abstraction is proposed.
A2	EX007–009 scanner and history induction	Two recursive tree-decoder proofs traverse the same fuel/tag/Option cases. Their quotation and depth conclusions become one joint invariant.
C1	EX057 direct physical selector sum; EX013 finite carrier positivity	A finite sum already known to be at most one yields all-zero or one selected index via a library characterization. No selector-word conversion is changed.
C2	EX047 half-radix identity; EX048 full-modulus geometry	Specialize an already proved `binaryWord_prefix` at $(1, 8)$ and $(2, 7)$; remove local function equalities and arithmetic presentation changes.
C3	EX048 geometric residue API; EX059 copied-field descriptor	Relocate one existing word-extensionality lemma so two earlier clients stop reopening finite sums. No direction/layout descriptor or shift theorem is introduced.
C4	EX007 scanner iteration mechanics	The same module already proves propagation from any cell. Zero-base propagation becomes a specialization, not a new iteration framework.
W1	EX058 chunk dispatch and sum assembly	A result indexed by the north color makes five newly added boundary classifications projections of one semantic case split. No 942-row dispatcher is changed.
S1	EX024 arithmetic DivMod certificate	One CRT-index proof names the exact congruence operations instead of search. The certificate and arithmetic interface are unchanged.
S2	EX032 degree profiles; EX020 syntax degree placement	Share only the repeated `Fin 2` composition-depth bookkeeping in five existing graph wrappers; retain every numerical ceiling.

2.1 Rejected rediscoveries

The inner recursor congruence proofs in `PRCode.Realizes.prec`, `ScalarPRCode.Realizes.prec` and `ScalarPROptimizer.eval_optimize` were considered and rejected: packet F already gives that change and the `RecursorCore.rec_congr` alternative (EX003). Likewise, this report does not re-propose sum-of-squares aggregation, shared circuit power/node certificates, nine-coordinate existence packaging, generic carry facts or source-bound profiles. Supporting audit scripts are reproducibility tools, not new proof-simplification ideas.

3 A1 Reuse total decoding for raw parent codes

Target. `AnnotatedParents.lean` lines 49–97, `check_raw_iff`. The theorem connects a numeric parent-list checker to an existential list of indices. Its forward proof reconstructs that list through a 44-line proof-body induction, despite the preceding `AnnotatedParents.lean` lines 29–45, `check_lists`, already establishing the exact typed-list characterization.

Change. Choose an encoded list using the existing raw-table decoder, rewrite to its exact recoding, then apply `check_lists`. The decoder returns arbitrary raw six-field rows; their row codes provide the needed natural-number list. This does not assert that the rows are valid proof lines.

```
constructor
· intro h
  obtain ⟨rows, hr⟩ :=
    AnnotatedRenameScan.decodeTable_total parents parents
      (Nat.le_refl _)
  have he : AnnotatedCodeArithmetic.listCode
      (rows.map AnnotatedRename.rowCode) = parents :=
    AnnotatedRenameScan.tableCode_of_decode hr
  refine ⟨rows.map AnnotatedRename.rowCode, he.symm, ?_⟩
  exact (check_lists fuel history expected _ hf).mp
    (by simpa only [he] using h)
· rintro ⟨indices, rfl, h⟩
  exact (check_lists fuel history expected indices hf).mpr h
```

This is the candidate proof body, line-wrapped for the page. The full statement and non-conflicting `check_raw_iff_short` declaration are in `annotated/AnnotatedCandidates.lean`. The exact conservative patch is `annotated/proposed.patch`; it has not been applied to the supplied source snapshot.

3.1 Why the bridge is available

`AnnotatedRenameScan.lean` lines 124–162 proves both `tableCode_of_decode` and `decodeTable_total`. Supplying `parents` as both code and sufficient external-natural fuel yields `rows`; exact recoding gives `he`. The parent-checker's forward direction is now the existing typed-list equivalence applied after that rewrite. The backward direction already has the encoded list, so it needs no decoder or reconstruction at all.

The import chain is already present. `AnnotatedParents` reaches `AnnotatedRenameScan` through `AnnotatedSchemaMaps` and `AnnotatedRenameScanPR`. There is no new import and no need for a new list-code injectivity theorem.

3.2 What must not be conflated

The decoder is used in a `Prop` proof; it is not inserted into the executable checker or its primitive-recursive program. The hypothesis `hf : expected.length ≤ fuel` and the checker fuel remain unchanged. The original source's emphasis on avoiding an unbounded decoder is not a reason to claim this dependency is free: the candidate deliberately uses an existing *bounded-decoder totality theorem* at a sufficient external-natural fuel. If the intended architecture forbids that proof dependency, keep the original induction even if this candidate elaborates.

Static outcome and gate. The whole declaration changes from 49 to 15 nonblank lines, and from 2,083 to 829 UTF-8 bytes, under the artifact's counting rule. First check implicit arguments and the proof-local `he` rewrite in the pinned Lean environment; then compare the old and new axiom dependency lists. No kernel-term or runtime reduction is claimed.

4 A2 Fuse quotation and depth decoder invariants

Targets. AnnotatedProofCode.lean lines 78–132: `Tree.decoded_depth_le` and `Tree.quote_of_decode`. Both traverse the same fuel, zero-code, tag and recursive `Option` cases. Only their terminal conclusions differ. One induction can return both facts, preserving the old names as projections.

```
theorem decode_spec {fuel code : Nat} {t : Tree}
    (h : decode fuel code = some t) :
    t.quote = code ∧ t.depth ≤ fuel := by
  induction fuel generalizing code t with
  | zero => simp [decode] at h
  | succ fuel ih =>
    cases code with
    | zero => simp [decode] at h
    | succ code =>
      have hp := Nat.pair_unpair code
      simp only [decode] at h
      cases he : (Nat.unpair code).1 with
      | zero =>
        simp only [he] at h hp
        cases h
        exact ⟨by simp only [quote, hp], by simp [depth]⟩
      | succ left =>
        simp only [he] at h hp
        cases ha : decode fuel left with
        | none => simp [ha] at h
        | some a =>
          cases hb : decode fuel (Nat.unpair code).2 with
          | none => simp [ha, hb] at h
          | some b =>
            simp only [ha, hb, Option.pure_def] at h
            cases h
            obtain ⟨hqa, hda⟩ := ih ha
            obtain ⟨hqb, hdb⟩ := ih hb
            exact ⟨by simp only [quote, hqa, hqb, hp],
              by simp only [depth]; omega⟩
```

The original `decoded_depth_le h` becomes `(decode_spec h).2`; `quote_of_decode h` becomes `(decode_spec h).1`. The fork branch uses the same two recursive successful decodes once, extracting their two quotation equalities and two depth bounds. The atom branch keeps `Nat.pair_unpair`; no numeric quotation, parser or depth definition changes.

Measured scope. Counting the combined theorem and both wrappers, the old pair's 54 nonblank declaration lines becomes 34; text changes from 1,723 to 1,304 UTF-8 bytes. The strengthened induction hypothesis does not strengthen any premise. The optional six-line `decode_eq_some_iff_short` combines this result with the existing `decode_quote`; it is included separately, not in the conservative patch or savings.

Gate. Test the branch simplification after `cases h`, the `Option.pure_def` normalization and the final maximum-depth arithmetic. Retain both old theorem statements, so clients need not change. This is a recursive tree-parser invariant, not the previously proposed scanner or history-fold framework.

5 C1 and C4 Replace special cases with existing facts

5.1 C1 Characterize a sum bounded by one

Target. CurrentCellLabels2011.lean lines 131–158, `sum_le_one_cases`. The current proof separately derives a nonzero summand, proves it equals one, removes it from the sum and shows every other summand vanishes. `Finset.sum_le_one_iff` already packages the decisive pairwise facts. It is used elsewhere in the pinned project, in `NativeSelectorClause54.selectors_one`.

```
theorem sum_le_one_cases {ι : Type*} [Fintype ι]
    (f : ι → ℕ) (hf : (∑ i, f i) ≤ 1) :
    (∀ i, f i = 0) ∨ ∃ i, Selected f i := by
  classical
  by_cases hz : ∀ i, f i = 0
  · exact Or.inl hz
  · push_neg at hz
    obtain ⟨i, hi⟩ := hz
    have hs := Finset.sum_le_one_iff.mp hf
    refine Or.inr ⟨i,
      (hs i i (Finset.mem_univ i) (Finset.mem_univ i) hi hi).2,
      ?_⟩
    intro j hji
    by_contra hj
    exact hji
      (hs j i (Finset.mem_univ j) (Finset.mem_univ i) hj hi).1
```

Preservation. Splitting on "every entry is zero" retains the empty index-type case without requiring a chosen element or a `Nonempty` instance. Only the second branch chooses a nonzero entry. The sum bound and `Selected` conclusion are unchanged. This is not a proposal to change how selector counts are decoded from the physical word.

Static delta. The proof body is 26 versus 10 nonblank lines in the candidate file (the listing above is rewrapped). Check the pinned mathlib signature and orientation of `sum_le_one_iff` before adoption.

5.2 C4 Make zero-base propagation a specialization

Targets. CurrentCellCycle2011.lean lines 113–122, `propagate`, and the existing CurrentCellCycle2011.lean lines 125–136, `propagate_from`. Move the latter before the former, then replace `propagate`'s body with:

```
  exact propagate_from P 0 hzero hnext
```

The general theorem already performs the same successor-preserved induction and uses `reachable_from`. Its proof does not depend on `propagate`, so reordering is acyclic. The predicate remains arbitrary; no word, occupation or tile hypothesis is added. The ambient `[NeZero (N * 2011)]` assumption and the single-cell case remain intact.

Tradeoff. Eight nonblank body lines become one, but the zero-base proof now uses general reachability rather than the simpler canonical `iterate_index_zero` fact. This is a dependency/readability decision, not automatically a faster proof. If the zero-base theorem is intentionally kept below the general reachability API, retain the independent proof.

6 C2 Normalize the two South prefix specializations

Targets. CurrentSouthShift2011.lean lines 92–140: `offset19_word` and `offset20_word`. Their generic engine, CurrentSouthShift2011.lean lines 30–47, `binaryWord_prefix`, already proves the needed low-prefix identity. The current specializations introduce a local function, prove a function equality by `funext`, rewrite a helper equality, and finish with a presentation conversion.

Change. First rewrite by `copy_word_translation`; then specialize the generic prefix theorem and normalize its numeric offsets, powers and association in a deliberately small `simpa only` set. No new arithmetic theorem is needed.

6.1 Offset 19

```
rw [copy_word_translation c t 941 b (by omega) hcopy]
simpa only [southWord, binaryWord_one, ← Nat.add_assoc,
  Nat.reduceAdd, Nat.reducePow, Nat.add_zero,
  Nat.cast_ofNat] using
  (binaryWord_prefix
    (fun j => c (((theta t b : Cell N) : ZMod (N * 2011)) +
      ((941 + j : ℕ) : ZMod (N * 2011))))
    1 8 (fun j _ => hc _))
```

The specialization is $k = 1, n = 8$: it shifts the nine-bit South word by one, reduces modulo $2^9 = 512$, and restores physical bit 941. The small `binaryWord_one` helper already exists privately in the target module.

6.2 Offset 20

```
rw [copy_word_translation c t 940 b (by omega) hcopy]
simpa only [southWord, binaryWord_two, ← Nat.add_assoc,
  Nat.reduceAdd, Nat.reducePow, Nat.add_zero,
  Nat.cast_ofNat] using
  (binaryWord_prefix
    (fun j => c (((theta t b : Cell N) : ZMod (N * 2011)) +
      ((940 + j : ℕ) : ZMod (N * 2011))))
    2 7 (fun j _ => hc _))
```

Here $k = 2, n = 7$: the multiplier is four, the modulus is again 512, and physical bits 940 and 941 form the two-bit prefix. The existing `binaryWord_two` helper supplies its expansion. Its duplication in the standalone candidate is only an import-access workaround for a private name; it is not an extra helper in the intended target edit.

6.3 Preserve the arithmetic contract

The exact hypotheses `hr : t % 2011 = 19` or `20`, the nine physical copy equalities, the Boolean bound and the full `ZMod (N * 2011)`) coordinates are retained. The candidate does not weaken a modulus, prove periodicity, alter a carry bound or invoke a half-radix identity. The generic prefix theorem only needs the retained suffix to be Boolean; the source specializations supply that from their existing global bound.

Static delta and gate. The respective bodies are 14 versus 6 and 16 versus 6 nonblank lines. Lean must check the exact normal forms under the pinned simp lemmas, especially the offset cast and left-associated addition. If the short normalization is brittle, retain a named offset equality rather than broaden the global simp set.

7 C3 Put binary word congruence beside its definition

Existing lemma. `CurrentTileColors2011.lean` lines 19–25 contains the right abstraction already:

```
theorem binaryWord_congr (f g : ℕ → ℕ) (n : ℕ)
    (h : ∀ j, j < n → f j = g j) :
    binaryWord f n = binaryWord g n := by
  unfold binaryWord
  apply Finset.sum_congr rfl
  intro j hj
  rw [h j (Finset.mem_range.mp hj)]
```

Two earlier clients reopen this implementation and manually lift equalities through the factor 2^j. Move the existing theorem into `CurrentCellLabels2011`, beside `binaryWord`, and use it at those clients. If namespace compatibility is required, keep a forwarding theorem under `CurrentTileColors2011.binaryWord_congr`.

7.1 Translate a copy word without reopening the sum

At `CurrentSouthShift2011.lean` lines 71–89, keep every hypothesis and use this body after the relocation:

```
  unfold southWord
  apply CurrentCellLabels2011.binaryWord_congr
  intro j hj
  have hs := source_eq_of_remainder_le b t (960+j) (by omega)
  rw [show 960+j-t%2011 = a+j by omega] at hs
  exact (hcopy ⟨j, hj⟩).trans (congrArg c hs)
```

The geometric equality `hs` remains the existing exact full-word shift fact. The only removed work is finite-range membership conversion and multiplication congruence. The standalone candidate references the lemma's *current* namespace because it imports the original modules; the target edit must use its new earlier location.

7.2 Aggregate horizontal bits in one application

At `CurrentOccupation2011.lean` lines 42–50, `horizontal_words`, the intended body is:

```
  exact CurrentCellLabels2011.binaryWord_congr _ _ 8
    (fun j hj => hbits ⟨j, hj⟩)
```

The exact direction remains East(current) equals West(previous). The congruence lemma is about arbitrary natural-valued functions; it adds no Boolean, occupation, carry or positivity premise.

7.3 The import direction is part of the change

Do not add a `CurrentTileColors2011` import to `CurrentSouthShift2011`. That would point back through the new physical word development and create a cycle. Hoisting the small theorem to the existing definition layer is the actual proposal, not merely changing the tactic invocation. The original theorem's proof needs only the definition and finite-sum API already available there.

Honest size accounting. The targeted bodies shrink from 10 to 6 and 4 to 2 nonblank lines. Moving the existing seven-line declaration does not remove it; a compatibility wrapper adds lines. Thus six removed client-body lines are not a guaranteed six-line whole-module saving. Adopt primarily for a stable abstraction boundary, then check the resulting import graph, all old theorem-name users and the affected Checks modules.

8 W1 Index the north boundary result by color

Targets. The five classifications at `WangTiles942Boundary.lean` lines 76–140 repeat the same six-way elimination from `WangTiles942Boundary.lean` lines 40–54, `high_north_cases`, then reject the other numeric colors. A small result indexed by the queried north color can centralize that elimination.

```
def NorthView (n : ℕ) (t : Tile) : Prop :=
  match n with
  | 434 => t = ⟨2,434,2,0⟩
  | 435 => t = start
  | 436 => t = ⟨3,436,4,4⟩ ∨ t = ⟨4,436,4,0⟩
  | 437 => ∃ a b : Fin 3, t = ⟨a.val^2,437,b.val,437⟩
  | 438 => ∃ p ∈ [434,435,436], t = ⟨1,438,1,p⟩
  | _ => False

theorem north_view {t : Tile} (ht : Allowed t)
    (hn : 434 ≤ t.north) : NorthView t.north t := by
  rcases high_north_cases ht hn with
    rfl | rfl | rfl | rfl | ⟨p,hp,rfl⟩ | ⟨a,b,rfl⟩
  · exact rfl
  · exact rfl
  · exact Or.inl rfl
  · exact Or.inr rfl
  · exact ⟨p,hp,rfl⟩
  · exact ⟨a,b,rfl⟩
```

The helper uses only the earlier high-north theorem, not any of the five classification theorems it would replace. It performs six semantic cases, not an enumeration of all 942 tile indices.

8.1 Keep the public classification statements

For example, the complete initializer theorem becomes:

```
theorem north_436_iff_initializers {t : Tile} (ht : Allowed t) :
    t.north = 436 ↔ t = ⟨3,436,4,4⟩ ∨ t = ⟨4,436,4,0⟩ := by
  constructor
  · intro hn
    simpa only [hn, NorthView] using north_view ht (by omega)
  · rintro (rfl | rfl) <;> rfl
```

The same forward body handles 434, 435, 437 and 438; their existing reflexive reverse directions remain. All five full candidates are in `wang/NorthViewCandidates.lean`.

Semantic checks. Color 436 keeps *both* initializers and the head initializer's south color 4, not the old state-93 color 188. Reset parameters remain in `[434,435,436]`; seam parameters remain `Fin 3`. The helper's lower bound is derived from the queried color, not added to a public theorem. No occupation, unique Start, periodicity, run or halting premise is introduced. The literal `Allowed` language and table remain untouched.

Optional priority. Old declarations total 60 nonblank lines; new wrappers total 34 and the helper adds 18, for 52 overall. Eight lines is a modest net saving. The value is one maintenance site for color-to-shape classification, not a large compression or performance claim. Keep the old proofs if the indirection is less readable. If adopted, use focused closed-color rewrites rather than globally adding the entire view to simp.

9 S1 and S2 Two small bounded quantification edits

9.1 S1 Name the CRT congruence already in the context

Target. `BoundedForallCertificate.lean` lines 87–91. The existing CRT-index branch unfolds `crtModulus`, invokes `gcongr` and ends with `exact`?. This is the only `exact`? occurrence among the 574 active Lean files at the pin. The branch's required fact is already `hIndex_residue k hk`.

The imported `MRDPCore` definition is `crtModulus scale index := (index + 1) * scale + 1`. Transport that residue through exactly those three operations:

```
· exact ((((hIndex_residue k hk).add_right 1).mul_right T).add_right 1).trans
    dvd_rfl.modEq_zero_nat
```

This two-line candidate replaces four tactic lines. This is a deterministic explicit proof term rather than a measured speedup. It does not change the certificate's witnesses, quantifiers, polynomial congruence, or the preceding falling-factorial branch. In particular, the truncated-subtraction handling using `min` is untouched. The exact patch and full replacement source are in `supplemental/`; the imported defining source has independently verified provenance in `supplemental/evidence/`.

9.2 S2 Pay the binary composition bookkeeping once

Targets. `BoundedForallDepth.lean` lines 81–120 repeats the same `Fin 2` elimination in `pow_depth_le`, `binomial_depth_le`, `sub_depth_le`, `mod_depth_le` and `div_depth_le`. The generic `GraphSizeBounds.lean` lines 91–111, `GraphRep.compose_depth_le`, gives $2n + 4 + h$; for $n = 2$ that is $h + 8$.

```
theorem compose_two_depth_le {σ : Type*}
    {f : (Fin 2 → ℕ) → ℕ} {g : Fin 2 → (σ → ℕ) → ℕ}
    (r : GraphRep f) (s : ∀ i, GraphRep (g i)) {h : ℕ}
    (hh : 1 ≤ h) (hr : r.depth ≤ h)
    (h0 : (s 0).depth ≤ h) (h1 : (s 1).depth ≤ h) :
    (r.compose s).depth ≤ h + 8 := by
  refine (GraphRep.compose_depth_le r s hh hr ?_).trans (by omega)
  intro i
  fin_cases i <;> assumption
```

For `pow_depth_le`, the candidate body is:

```
  unfold GraphRep.pow
  exact compose_two_depth_le _ _ (by omega)
    (BoundedForallDepth.pow_depth.trans hh) hr hs
```

Limited recommendation. The artifact supplies the helper and two pilots, not all five replacements. Each client body is four versus two nonblank lines in its source formatting. Extending to all five would remove ten client-body lines, but the nine-line helper offsets nine of them; its docstring eliminates even that nominal whole-file gain. Include this only if a single maintenance point for further binary operations is valuable. All graph constructors, witness counts, fixed 512 ceilings and +8 bounds stay unchanged. This is neither a new bound nor a representation redesign.

10 Measurements and acceptance gates

10.1 Separate measured text from proposed maintenance benefits

All counts below come from pinned source and candidate text, without a formatter. "Declaration" includes its header; "body" starts after `:= by`. Empty lines are excluded. Listings in this report may wrap differently. These are not proof-term size, elaboration time, runtime or kernel-memory measurements.

ID	Measured unit	Kind	Old	New
A1	`check_raw_iff`	Declaration	49	15
A2	Two decoder facts, shared theorem and wrappers	Declarations	54	34
C1	`sum_le_one_cases`	Body	26	10
C2	`offset19_word`	Body	14	6
C2	`offset20_word`	Body	16	6
C3	`copy_word_translation`	Body	10	6
C3	`horizontal_words`	Body	4	2
C4	`propagate`	Body	8	1
W1	Five classifications, including helper overhead	Declarations	60	52
S1	One CRT branch	Tactic lines	4	2
S2	Each binary-depth pilot	Body	4	2

C3's compatibility alias is additional overhead; its client-body saving is not a whole-project net figure. S2 is not a compression headline. A1/A2's optional complete decoder iff is excluded from their count. No aggregate "percentage faster" or total verified reduction follows from this table.

10.2 The validation sequence

1. **Use the actual pinned project.** Work in a complete disposable checkout at the review pin, with `leanprover/lean4:v4.32.0` and the locked mathlib dependencies. The selected source snapshot is evidence, not a standalone Lake workspace.
2. **Compile the independent candidates first.** Record Lean/Lake versions and command output. Check A1's total-decoder dependency, A2's recursive option simplification, C1's library signature and C2's numeric normal forms before editing production proofs.
3. **Test the actual edit shape.** Apply exact patches only in the throwaway checkout. For C3, relocate the helper and retain required aliases; for C4, reorder declarations. Compiling a standalone file that imports later modules does not validate these production import-order changes.
4. **Compare public statements and axioms.** Preserve old theorem names, hypotheses and conclusions; inspect every expected endpoint's `#print axioms` result. A missing or empty audit log is not a pass. Compare dependency sets with the original declarations rather than merely searching for one forbidden string.
5. **Replay affected consumers.** Rebuild the annotated checker, current word/cell/occupation modules, Wang boundary/seam consumers and bounded-quantification Checks modules as applicable. Only then measure elaboration costs with identical options and a recorded warm/cold policy.

Available evidence. Byte verification, static declaration checks, patch dry runs and finite arithmetic smoke checks help catch packaging or specification mistakes. None proves that Lean accepts a candidate. The supporting `validation/` tools exist to make future replay reproducible; they are not another novelty claim or a substitute for it.

11 Review corpus and reusable artifacts

11.1 The six prior packets

The 75 scoped exclusions in `incoming_audit/EXCLUSION_CATALOG.md` refer to these manuscript sources. Full Git blob identities and SHA-256 values are in the accompanying manifest; all six archive byte streams were verified. The retired packet's placed TeX/PDF match its archive byte for byte.

A
`Lean_Proof_Simplification_2026-10-26.zip`. Manuscript: `Lean_Proof_Simplification/report/report.tex`.
B
`ProveIt_Lean_Proof_Simplification_2026-10-26.zip`. Manuscript: `LeanReview/report/proof_simplification_report.tex`.
C
`diophantine-lean-simplification-2026-10-26.zip`. Manuscript: `diophantine-lean-simplification-2026-10-26/report.tex`.
D
`lean-proof-simplification-2026-10-26 (2).zip`. Manuscript: `lean-proof-simplification-2026-10-26/report.tex`.
E
`lean-proof-simplification-2026-10-26.zip`. Manuscript: `lean-proof-simplification-2026-10-26/report.tex`.
F
Retired `diophantine-lean-proof-simplification-2026-10-26.zip`, received at commit `151a732b91`. Placed manuscript at the review pin: `research/lean-proof-simplification-20261026/report.tex`.

11.2 What to use from the new archive

- `report/`: this editable TeX source, PDF, build script and QA record.
- `annotated/`, `current/`, `wang/`, `supplemental/`: complete uncompiled candidates, exact patches where available, explanations and reproducible source-text measurements.
- `incoming_audit/`: exclusion catalog, explicit novelty decisions, archive identities and audit results, without repackaging the old reports as new work.
- `provenance/`, `sources/`, `supplemental/evidence/`: immutable source anchors, byte manifests, recent-commit evidence and selected source material. Archive contents are evidence, not a replacement checkout.
- `validation/`: future replay plan, non-vacuous endpoint/axiom checks and static validation tools. Read its instructions before running it.

11.3 Decisions that remain after compilation

Start with C4, S1, C1 and C2. Then evaluate A2, and A1 if its existing bounded-decoder dependency is acceptable. C3 requires the correct import placement. W1 and S2 are optional maintenance choices: compilation alone does not make their abstraction worthwhile.

Before adoption, establish that the candidates elaborate under the pinned dependencies, public types and axiom sets meet project policy, and the edits improve readability. Novelty is relative to the six reviewed packets, not a claim of global originality. No accepted simplification or performance result is asserted before that replay.